

Non-Linear Data Structures

June 29, 2026

Content

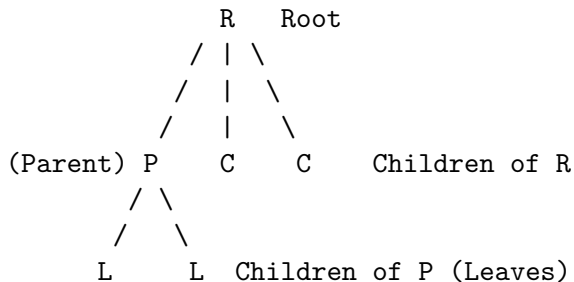
- ▶ Non-Linear Data Structures
- ▶ Trees and Basics
- ▶ Binary Tree
- ▶ Tree Implementation
 - Array based Tree
 - List based Tree
- ▶ Priority Queues
 - List based Priority Queue
 - Heaps
- ▶ Dictionaries and Maps
 - Hash Table using Array
 - Collision Handling
- ▶ Bloom Filter and Variants
- ▶ Skip Lists

Non-Linear Data Structures

- ▶ A Non-Linear Data Structure is a way of organizing data where elements are not arranged sequentially.
- ▶ Each element can connect to multiple other elements, forming hierarchical or network relationships.
- ▶ Unlike linear structures (arrays, linked lists, stacks, queues), data is not stored in a single line.

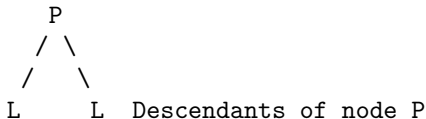
Trees and Basics

- ▶ A tree is an abstract data type that stores elements hierarchically.
- ▶ *Basics of Tree:*
 - *Root (R)*: Node in a tree with no parent
 - *Parent (P)*: Node in a tree which has multiple nodes connected as ascendants
 - *Children (C)*: Nodes in a tree which have a parent
 - *Leaves (L)*: Nodes in a tree with no children
 - *Branch (Edge)*: A connection between parent and children, e.g., (R, P), (R, C)



Trees and Basics

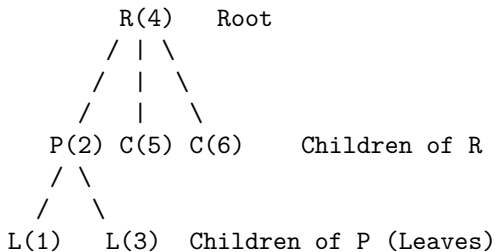
- *Sub Tree (R)*: Node with all its descendants of a tree



- *Path*: A connection between two nodes formed through sequence of edges, e.g., path $(R, L) - \{(R, P), (P, L)\}$
- *Depth*: Depth of a tree from a node is number of ancestors excluding the same node, e.g., Depth w.r.t Node L is 2 (refer picture at previous slide)
- *Height*: Height of a tree from a node is maximum of the path lengths from the node to any of the descendant leaf, e.g., Height w.r.t Node R is 2 and Height w.r.t Node P is 1

Tree Traversal

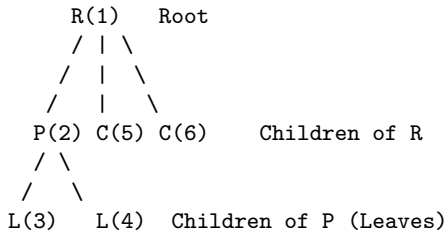
- ▶ *Depth first (DFS)*: The traversal takes order top-to-bottom and bottom-to-top
 - *Inorder*: Left Sub tree → Root → Right Sub tree



Tree Traversal

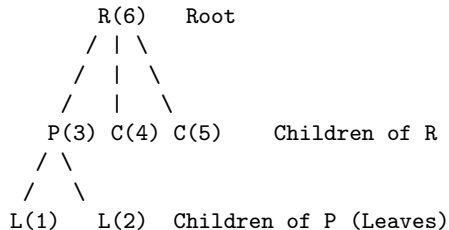
Preorder:

Root → Left Tree → Right Tree



Postorder:

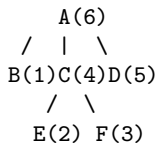
Left Tree → Right Tree → Root



Tree Traversal

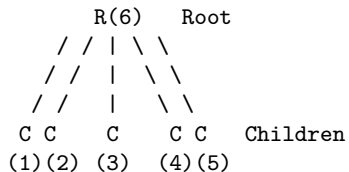
Postorder:

Left Tree $\rightarrow$ Right Tree $\rightarrow$ Root



Postorder:

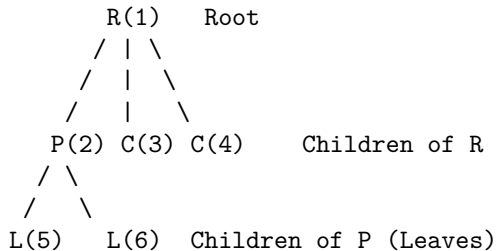
Left Tree $\rightarrow$ Right Tree $\rightarrow$ Root



Tree Traversal

- ▶ *Breadth first (DFS)*: The traversal takes order left-to-right

- Root → Left Sub tree → Right Sub tree



Binary Tree

- ▶ In a binary tree, each node has at most 2 children:
 - left child
 - right child

```
1 typedef struct Node {  
2     char data;  
3     struct Node* left;  
4     struct Node* right;  
5 } Node_t;
```

Binary Tree

```
1 Node_t* createNode(char data) {
2     Node_t* newNode = (Node_t*)malloc(sizeof(Node_t));
3
4     newNode->data = data;
5     newNode->left = NULL;
6     newNode->right = NULL;
7
8     return newNode;
9 }
10
11 void postorder(Node_t* root) {
12     if (root == NULL)
13         return;
14
15     postorder(root->left);    // Left subtree
16     postorder(root->right);   // Right subtree
17     printf("%c", root->data);
18 }
```

Binary Tree

```
1 Node_t* A = createNode('A');
2 Node_t* B = createNode('B');
3 Node_t* C = createNode('C');
4 Node_t* D = createNode('D');
5 Node_t* E = createNode('E');
6 Node_t* F = createNode('F');
7 Node_t* G = createNode('G');
8
9
10 A->left = B;
11 A->right = C;
12 B->left = D;
13 B->right = E;
14 C->left = F;
15 C->right = G;
16
17 postorder(A);
```

Binary Tree

```
1 - postorder(A)
2     - postorder(B)
3         - postorder(D) (1) (prints)
4         - postorder(E) (2) (prints)
5     - postorder(B) (3) (prints left sub tree root)
6     - postorder(C)
7         - postorder(F) (4) (prints)
8         - postorder(G) (5) (prints)
9     - postorder(C) (6) (prints right sub tree root)
10 - postorder(A) (7) (prints tree root)
```

Binary Tree

```
1 void preorder(Node_t* root) {
2     if (root == NULL)
3         return;
4
5     printf("%c", root->data);
6     preorder(root->left);    // Left subtree
7     preorder(root->right);   // Right subtree
8
9 }
10
11 - preorder(A) (1)           (prints root)
12     - preorder(B) (2)       (prints left child of A)
13         - preorder(D) (3)   (prints left child of B)
14         - preorder(E) (4)   (prints right child of B)
15     - preorder(C) (5)       (prints right child of A)
16         - preorder(F) (6)   (prints left child of C)
17         - preorder(G) (7)   (prints right child of C)
```

Binary Tree

```
1 void inorder(Node_t* root) {
2     if (root == NULL)
3         return;
4
5     inorder(root->left);    // Left subtree
6     printf("%c", root->data);
7     preorder(root->right);  // Right subtree
8
9 }
10 - inorder(A)
11     - inorder(B)
12         - inorder(D) (1)          (prints left child of B)
13     - inorder(B) (2)              (prints B)
14         - inorder(E) (3)          (prints right child of B)
15 - inorder(A) (4)                  (prints root A)
16     - inorder(C)                  (prints right child of A)
17         - inorder(F) (5)          (prints left child of C)
18     - inorder(C) (6)              (prints C)
19         - inorder(G) (7)          (prints right child of C)
```

Binary Tree - Breadth First

```
1 // Tree Node Structure
2 typedef struct Node {
3     char data;
4     struct Node* left;
5     struct Node* right;
6 } Node_t;
7
8 // Queue Structure
9 typedef struct QNode {
10     Node_t* TNode;
11     struct QNode* next;
12     struct QNode* front;
13     struct QNode* rear;
14 } QNode_t;
```

Binary Tree

```
1 Node_t *root = createNode('A'); root->left = createNode('B');
2 root->right = createNode('C')); root->left->left = createNode('D');
3 root->left->right = createNode('E'); root->right->left = createNode('F')));
4 root->right->right = createNode('G'));
5 QNode_t* top = NULL;
6 enqueue(top, root);      \\ Root node queuing
7 void breadthFirstTraversal(Node_t *root) {
8     if (root == NULL)
9         return;
10    while (top->front != NULL) {
11        Node_t *current = dequeue(top); // Empty after Dequeue
12        printf("%d", current->data);    // Print Root "A"
13        if (current->left != NULL)
14            enqueue(top, current->left);
15        if (current->right != NULL)
16            enqueue(top, current->right);
17    }
18 }
```

Recap - Enqueue

```
1 void enqueue(QNode_t* top, Node_t* treeNode)
2 {
3     QNode_t* queueNode = (QNode_t*)malloc(sizeof(QNode_t));
4     queueNode->TNode = treeNode;
5     queueNode->next = NULL;
6     if (top == NULL)          {           \\ empty queue
7         top = queueNode;
8         top->front = queueNode; \\ first node as front
9         top->rear = top->front; \\ first node as rear
10        return;
11    }
12    top->rear->next = queueNode;           \\ Newly Enqueued
13    top->rear = queueNode;                 \\ New rear = enqueued one
14 }
```

Recap - Dequeue

```
1 Node_t* dequeue(QNode_t *q) {  
2     if (q->front == NULL)           // Empty Queue  
3         return NULL;  
4  
5     QNode_t *temp = q->front;  
6     Node_t *node = temp->TNode;      // BFS Tree Node for return  
7     q->front = q->front->next;       // New Front Node  
8     if (q->front == NULL)           // No more queue after dequeue  
9         q->rear = NULL;  
10  
11     free(temp);                     // Free the old front  
12  
13     return node;  
14 }
```

Binary Tree

- ▶ Iteration 1: a.) Add parent 'A' (root) into the queue, b.) Dequeue, c.) print/access, d.) Enqueue its children 'B' and 'C' into queue
- ▶ Iteration 2: a.) Dequeue the left child 'B', b.) print/access, c.) Enqueue its children 'D' and 'E' into queue
- ▶ Iteration 3: a.) Dequeue the right child 'C', b.) print/access, c.) Enqueue its children 'F' and 'G' into queue
- ▶ Pattern goes till the queue becomes empty

Binary Tree Using Arrays

```
1 #define SIZE 100
2 int tree[SIZE];
3 for(int i = 0; i < SIZE; i++) {
4     tree[i] = -1;
5 }
6 void insertRoot(int value) {
7     if(tree[0] == -1) tree[0] = value;
8 }
9 void insertLeft(int parentIndex, int value) {
10     int left = 2 * parentIndex + 1;
11     if(tree[parentIndex] == -1) return;
12     tree[left] = value;
13 }
14 void insertRight(int parentIndex, int value) {
15     int right = 2 * parentIndex + 2;
16     if(tree[parentIndex] == -1) return;
17     tree[right] = value;
18 }
```

Binary Tree Using Array

```
1 insertRoot(1);  
2  
3 insertLeft(0, 2);  
4 insertRight(0, 3);  
5  
6 insertLeft(1, 4);  
7 insertRight(1, 5);  
8  
9 insertLeft(2, 6);  
10 insertRight(2, 7);
```

Priority Queue - Sorted Queue

- ▶ *Priority Queue*: Inserts new node such that the values in each node will be in sorted order.
- ▶ Simply following append or insert at end can't give sorted order. Queue's enqueue method is modified

Priority Queue - Sorted Queue

```
1 Node_t* front = NULL;
2 void enqueue(int value) {
3     Node_t *newNode = (Node_t*)malloc(sizeof(Node_t));
4     newNode->data = value;
5     newNode->next = NULL;
6     // smallest: value < front->data (insert as front)
7     if(front == NULL || value < front->data) {
8         newNode->next = front;
9         front = newNode;
10    }
11    else {
12        Node_t *temp = front;
13        while(temp->next != NULL && temp->next->data < value) {
14            temp = temp->next;
15        }
16        newNode->next = temp->next;
17        temp->next = newNode;
18    }
19 }
```

Priority Queue - Sorted Queue

```
1 void dequeue() {  
2     if(front == NULL) return;  
3     Node_t *temp = front;  
4     front = front->next;  
5     free(temp);  
6 }
```

Priority Queues Using Heaps

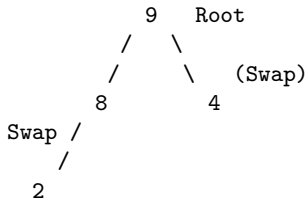
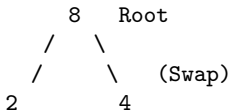
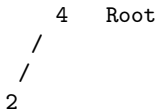
- ▶ *Heap*: A specialized tree-based data structure that satisfies the heap property. It's commonly used to implement priority queues efficiently.
- ▶ *Heap Property*:
 - *Min Heap*: Parent Value less than Children Values throughout tree
 - *Max Heap*: Parent Value greater than Children Values throughout tree

```
1 #define MAX 100
2
3 int heap[MAX];
4 int size = 0;
```

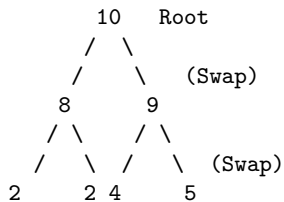
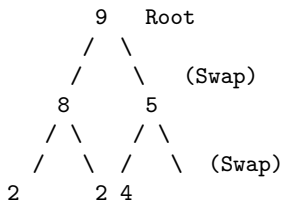
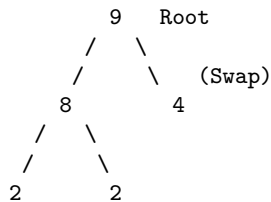
Priority Queues Using Heaps

MAX HEAP

T = {4, 2, 8, 9, 2, 5, 10}



Priority Queues Using Heaps



MAX HEAPED TREE $T = \{10, 8, 9, 2, 2, 4, 5\}$

Priority Queues Using Heaps

```
1 // MIN HEAP
2 void insert(int value) {
3     heap[size] = value;
4     int i = size;
5     size++;
6     // From second insert
7     while (i > 0) {
8         // 'i' is child, Tree using Array (Refer)
9         int parent = (i - 1) / 2;
10        if (heap[parent] > heap[i]) {
11            swap(&heap[parent], &heap[i]);
12            i = parent;
13        } else {
14            break;
15        }
16    }
17 }
```

Dictionaries and Maps

- ▶ *Hash Table*: A hash table is a data structure that stores data in key - value pairs and provides very fast:
 - insertion
 - searching
 - deletion
- ▶ Hash Table uses the hash function to convert a key into an array index.

```
1 int hash(int key)    // Hash function
2     return key % 10; // remainder - value between 0 - 9
3 int key = 15;
4 int idx = hash(key);
```

Hash Table Using Array

```
1 #define SIZE 10
2 typedef struct HashItem {
3     int key;
4     int value;
5 } Hash;
6 Hash hashTable[SIZE];
7 void initialize() {
8     for(int i = 0; i < SIZE; i++) {
9         hashTable[i].key = -1;
10        hashTable[i].value = -1;
11    }
12 }
13 int hashFunction(int key) {
14     return key % SIZE;
15 }
```

Hash Table Using Array

```
1 void insert(int key, int value) {
2     int index = hashFunction(key);
3     hashTable[index].key = key;
4     hashTable[index].value = value;
5 }
6 int search(int key) {
7     int index = hashFunction(key);
8     if(hashTable[index].key == key)
9         return hashTable[index].value;
10    return -1;
11 }
12 // Main
13 initialize();
14 insert(1, 100); insert(2, 200); insert(3, 300);
15 printf("Key1=>Value:%d\n", search(1));
16 printf("Key2=>Value:%d\n", search(2));
17 printf("Key3=>Value:%d\n", search(3));
```

Hash Table Using Array

- ▶ Hash Table uses the hash function to convert a key into an array index.

```
1 int hash(int key)    // Hash function
2     return key % 10; // remainder - value between 0 - 9
3 int key = 15;
4 int idx = hash(key);
```

- ▶ $\text{hash}(15) == \text{hash}(25)$ (called collision)
- ▶ *Chaining* - collision handling by storing values in linked list fashion

Hash Table Using Array

```
1 #define SIZE 10
2 typedef struct Node {
3     int key;
4     int value;
5     struct Node* next;
6 } Hash;
7 Hash* hashTable[SIZE];
8 int hashFunction(int key) {
9     return key % SIZE;
10 }
11 void insert(int key, int value) {
12     int index = hashFunction(key);
13     Hash* newNode = (Hash*)malloc(sizeof(Hash));
14     newNode->key = key;
15     newNode->value = value;
16     newNode->next = hashTable[index]; // Insertion at head
17     hashTable[index] = newNode;      // Insertion at head
18 }
```

Hash Table Using Array

```
1 int search(int key) {
2     int index = hashFunction(key);
3     Hash* temp = hashTable[index];
4     while(temp != NULL) {
5         if(temp->key == key)
6             return temp->value;
7         temp = temp->next;
8     }
9     return -1;
10 }
11 // Main
12 insert(1, 100);
13 insert(11, 200);
14 insert(21, 300);
15 printf("\nSearch11=%d\n", search(11));
```

Hash Table Using Array

- *Linear Probing*: Search for available index to fill the value

```
1 void insert(int key) {
2     int index = hashFunction(key);
3     while(hashTable[index] != -1) {
4         index = (index + 1) % SIZE;
5     }
6     hashTable[index] = key;
7 }
8 int search(int key) {
9     int index = hashFunction(key);
10    int start = index;
11    while(hashTable[index] != -1) {
12        if(hashTable[index] == key) return index;
13        index = (index + 1) % SIZE;
14        if(index == start) break; // No Match Found
15    }
16    return -1;
17 }
```

Bloom Filter

- ▶ A Bloom filter is a data structure used to test whether an element is in a set.
 - Definitely not present
 - Possibly present
- ▶ It is extremely fast and memory efficient.
- ▶ A Bloom filter uses:
 - a bit array
 - multiple hash functions
- ▶ Insert
 - Run multiple hash functions
 - Each hash gives an index
 - Set those bit positions to 1
- ▶ Search
 - Compute the same hashes
 - Check those bit positions
 - If any bit is 0 → definitely NOT present
 - If all bits are 1 → possibly present

Bloom Filter

```
1 #define SIZE 100
2 int bloom[SIZE];           // Intitalize to zero using memset
3 int hash1(char *str) {     // Hash Fn 1
4     int hash = 0;
5     while(*str)
6         hash += *str++; // Hash Key1 - ASCII sum of string chars
7     return hash % SIZE;   // Index1
8 }
9 int hash2(char *str) {     // Hash Fn 2
10    int hash = 1;
11    while(*str)
12        hash = (hash * 31) + *str++; // Hash Key2
13    return hash % SIZE;     // Index2
14 }
15 void insert(char *str) {
16     int h1 = hash1(str);
17     int h2 = hash2(str);
18     bloom[h1] = 1; bloom[h2] = 1; // Insert bit '1' at both indices
19 }
```

Bloom Filter

```
1 // Search
2 int search(char *str) {
3     int h1 = hash1(str);
4     int h2 = hash2(str);
5     if(bloom[h1] && bloom[h2]) return 1;
6     return 0;
7 }
8 // Main
9 insert("apple");
10 insert("banana");
11 if(search("grape"))
12     printf("Possibly-Present\n");
13 else
14     printf("Not-Present\n");
```

Counting Bloom Filter

- ▶ A standard Bloom filter cannot delete elements.
- ▶ Replace bits with counters to delete and insert.

```
1 void insert(char *str) {  
2     int h1 = hash1(str);  
3     int h2 = hash2(str);  
4     bloom[h1]++;    bloom[h2]++;    // Insert bit '1' at both indices  
5 }  
6 int search(char *str) {  
7     int h1 = hash1(str);  
8     int h2 = hash2(str);  
9     if(bloom[h1] > 0 && bloom[h2] > 0) return 1;  
10    return 0;  
11 }
```

Scalable Bloom Filter

- ▶ A standard Bloom Filter has a fixed size:
 - if too many elements are inserted, false positives increase rapidly.
 - Example: `bloom[0]++` and `bloom[5]++` for two keys (more insert scenario). Assume `key1` bloom(s) decremented. If search is executed with key 1, it will display "Possibly present", but actually not-present (False positive)
- ▶ Scalable Bloom Filters solve this by adding new Bloom Filters when the current one becomes too full.

Scalable Bloom Filter

- ▶ Let us start with 1st bloom filter of 1024 bit places for insert
- ▶ With this filter, the capacity of $0.1 * 1024 \approx 102$ inserts allowed.
- ▶ If insert counter exceeds capacity, New Bloom Filter will be created of size $2 * 1024$ bit places
- ▶ With this filter, the capacity is $0.1 * 1024 * 2 \approx 204$ inserts allowed.
- ▶ If insert counter exceeds capacity, New Bloom Filter will be created of size $4 * 1024$ bit places
- ▶ The process repeats till 10 Bloom filters creation.

Scalable Bloom Filter

```
1 #define INITIAL_SIZE 1024
2 #define MAX_FILTERS 10
3 // ----- Bloom Filter -----
4 typedef struct BloomFilter{
5     char* bitset;
6     int size;
7     int count;
8     int capacity;
9 } BloomFilter;
10 BloomFilter* create_filter(int size) {
11     BloomFilter* bf = malloc(sizeof(BloomFilter));
12     bf->size = size;
13     bf->count = 0;
14     bf->capacity = size * 0.1; // threshold (10% fill)
15     bf->bitset = calloc(size, sizeof(char)); // BF Array with '0's
16     return bf;
17 }
```

Scalable Bloom Filter

```
1 typedef struct ScalableBloom {
2     BloomFilter **filters;
3     int num_filters;
4 } ScalableBloom;
5 ScalableBloom* create_sbf() {
6     ScalableBloom *sbf = malloc(sizeof(ScalableBloom));
7     sbf->filters = malloc(sizeof(BloomFilter*) * MAX_FILTERS);
8     sbf->filters[0] = create_filter(INITIAL_SIZE);
9     sbf->num_filters = 1;
10    return sbf;
11 }
12 int hash_fn(const char *str) { // Hash Fn.
13     int hash = 2166136261u;
14     while (*str) {
15         hash ^= (*str++);
16         hash *= 16777619;
17     }
18     return hash;
19 }
```

Scalable Bloom Filter

```
1 void sbf_add(ScalableBloom *sbf, const char *item) {
2     BloomFilter *bf = sbf->filters[sbf->num_filters - 1]; // recent add
3     if (bf->count >= bf->capacity) { // Limit exceeds, New BF
4         if (sbf->num_filters >= MAX_FILTERS) return;
5         int new_size = bf->size * 2;
6         sbf->filters[sbf->num_filters++] = create_filter(new_size);
7         bf = sbf->filters[sbf->num_filters - 1];
8     }
9     bf->bitset[hash_fn(item) % bf->size] = 1;
10    bf->count++;
11 }
12 int sbf_check(ScalableBloom *sbf, const char *item) {
13     int h = hash_fn(item);
14     for (int i = 0; i < sbf->num_filters; i++) {
15         BloomFilter* bf = sbf->filters[i];
16         if (bf->bitset[h % bf->size]) return 1;
17     }
18     return 0;
19 }
```

Compressed Bloom Filter

```
1 int hash(const char *s) {           // Hash Func.
2     int h = 5381;
3     while (*s) h = ((h << 5) + h) + *s++;
4     return h % SIZE;
5 }
6 int compress_bf(char *in, int in_size, char *out, int *out_size) {
7     return compress(out, out_size, in, in_size); // zlib.h
8 }
```

- ▶ searches repeated patterns
- ▶ encodes symbols ($ABCDE|ABC \rightarrow (ABC, (3, 5))$) - 3 char ABC repeats, previous instance 5 chars before (Refer Lempel-Ziv Compression Techniques (LZ77))
- ▶ writes packed bitstream

```
1 // Compile: gcc Compressed_Bloom_Filter.c -o Compressed_Bloom_Filter -lz
2 compress_bf(bf, sizeof(bf), compressed, &comp_size); // less storage size
3 decompress_bf(compressed, comp_size, restored, &restored_size);
4 // search after decompress
```

Compressed Bloom Filter

```
1 // ----- decompress -----  
2 int decompress_bf(char *in, int in_size, char *out, int *out_size) {  
3     return uncompress(out, out_size, in, in_size);  
4 }
```

- ▶ decodes symbols $((ABC, (3, 5)) \rightarrow ABC - -|ABC)$ - just for example, skipped the decoding part of *DE*
- ▶ writes packed bitstream

Partitioned Bloom Filter

- ▶ *Partitioned Bloom Filter*: A variation of a standard Bloom Filter where the bit array is divided into separate partitions, and each hash function maps into exactly one partition.
- ▶ Suppose:
 - Bit Array Size, 32
 - Number of Partitions, 4
 - Partitions are
 - Partition 1 : bits 0-7
 - Partition 2 : bits 8-15
 - Partition 3 : bits 16-23
 - Partition 4 : bits 24-31

Generated hash keys for each partition from string "apple" sets bit '1' at a supported index returned by respective hash fns.

$h1(\text{"apple"}) \rightarrow 3$ (inside partition 1)
 $h2(\text{"apple"}) \rightarrow 10$ (inside partition 2)
 $h3(\text{"apple"}) \rightarrow 18$ (inside partition 3)
 $h4(\text{"apple"}) \rightarrow 29$ (inside partition 4)

Partitioned Bloom Filter

```
1 // Different seed, different hash function
2 #define K      (4)
3 #define M      (1024)
4 typedef unsigned int uint;
5 uint hash_i(const char *str, uint seed) {
6     int hash = 2166136261u ^ seed;  \\ XOR
7     while (*str) {                  \\ FNV-1a
8         hash ^= *str++;
9         hash *= 16777619;
10    }
11    return hash;
12 }
```

Partitioned Bloom Filter

```
1 char filter[K][M];
2 void insert(const char *key) {
3     for (int i = 0; i < K; i++) {
4         uint32_t h = hash_i(key, i);
5         int idx = h % M;
6         filter[i][idx] = 1;
7     }
8 }
9 int contains(const char *key) {
10     for (int i = 0; i < K; i++) {
11         uint h = hash_i(key, i);
12         int idx = h % M;
13         if (filter[i][idx] == 0)
14             return 0; // definitely not present
15     }
16     return 1; // possibly present
17 }
```

Skip Lists

- ▶ *Skip Lists*: Ordered data structure that works like a multi-level linked list, allowing fast search, insertion, and deletion—similar in performance to a balanced tree but much simpler to implement.
- ▶ It's a key structure in systems like ordered indexes.
- ▶ Steps:
 - Create a skip list with N linked list heads (level 1)
 - For insertion, randomly select the number of levels L and fill the same value as first node value in all levels till L
 - Keep the inserted nodes as head nodes for L levels
 - For next insertion, traverse and find the place to be inserted at each of the L levels in sorted manner.
 - Randomly select the number of levels L_1 and insert the new value in a sorted manner for all levels till L_1
 - Still, the global level will be the maximum one.

Skip Lists

```
1 #define MAX_LEVEL 6      // Maximum Number of Levels for each Node
2 typedef struct Node {
3     int value;
4     struct Node* forward[MAX_LEVEL]; // Array of Pointers (Levels)
5 } Node_t;
6 typedef struct SkipList {
7     int level;
8     Node_t *header;
9 } SkipList;
```

Skip Lists

```
1 Data Sequence: 10, 8, 7
2 Random Value: 4, 2, 3
3
4 Level 3: 10
5 Level 2: 7 10
6 Level 1: 7 8 10
7 Level 0: 7 8 10
8
9 Level:      3  2  4  (Random Value)
10 Level 3:  -  - 10  (Skip List)
11 Level 2:  7  - 10
12 Level 1:  7  8 10
13 Level 0:  7  8 10
```

Skip Lists

```
1 SkipList* init() {  
2     Node_t* n = (Node_t*)malloc(sizeof(Node_t));  
3     n->value = -1;;  
4     for (int i = 0; i < MAX_LEVEL; i++)  
5         n->forward[i] = NULL;  
6     SkipList *list = malloc(sizeof(SkipList));  
7     list->level = 1;  
8     list->header = n;  
9     return list;  
10 }
```

Skip Lists

```
1  int random_level() {
2      int level = 1;
3      // Fix probability P to a value between 0 & 1
4      while (((double)rand() / RAND_MAX) < P &&
5             level < MAX_LEVEL)
6      {
7          level++;
8      }
9      // Number of while loop iterations: 5, level - 6
10     // Number of while loop iterations: 4, level - 5
11     // Number of while loop iterations: 3, level - 4
12     // Number of while loop iterations: 2, level - 3
13     // Number of while loop iterations: 1, level - 2
14     // Number of while loop iterations: 0, level - 1
15     return level;
16 }
```

Skip Lists - 1st insert

```
1 void insert(SkipList *list, int value) { // value = 10
2     Node_t* update[MAX_LEVEL];
3     // 1st insert- value (-1), forward (NULL)
4     Node_t* curr = list->header;
5     update[0] = curr; curr = curr->forward[0]; // NULL
6     int lvl = random_level(); // random level = 4
7     // for 1st insert, true (4 > 1)
8     if (lvl > list->level) {
9         for (int i = list->level; i < lvl; i++) // i= 1:3
10             update[i] = list->header;
11         // level: 1 + 3
12         list->level = lvl;
13     }
14     Node_t *n = (Node_t*)malloc(sizeof(Node_t));
15     n->value = value; // update the new value
16     for (int i = 0; i < lvl; i++) {
17         n->forward[i] = update[i]->forward[i]; // points NULL
18         update[i]->forward[i] = n; // points new node
19     }
```

Skip Lists - 2nd insert

```
1 void insert(SkipList *list, int value) { // value = 8
2     Node_t* update[MAX_LEVEL];
3     // 2nd insert- header initial value (10)
4     Node_t* curr = list->header;
5     for (int i = list->level - 1; i >= 0; i--) { // i: 3:0
6         while (curr->forward[i] && curr->forward[i]->value < value)
7             { // Traverse & find insert position based on ascend. order
8                 curr = curr->forward[i];
9             }
10        update[i] = curr;
11    }
12    curr = curr->forward[0];
13    if (curr && curr->value == value) return;
14
15    // continues
```

Skip Lists- 2nd insert

```
1      // assume random levels (2) less than 4
2      int lvl = random_level();
3      // for 2nd insert, false (2 > 4), skip the if block
4      if (lvl > list->level) {
5          // do nothing
6      }
7      Node_t *n = (Node_t*)malloc(sizeof(Node_t));
8      n->value = value; // update the new value
9      // For first two levels, update the nodes
10     for (int i = 0; i < lvl; i++) {
11         // newNode points to Node, pointed by curr. node
12         n->forward[i] = update[i]->forward[i];
13         // current node should point newNode
14         update[i]->forward[i] = n;
15     }
16 }
```

Skip Lists Hierarchy till 2nd insert

```
1 Level 3: 10
2 Level 2: 10
3 Level 1:  8 10
4 Level 0:  8 10
```

Skip Lists - 3rd insert

```
1 void insert(SkipList *list, int value) { // value = 7
2     Node_t* update[MAX_LEVEL];
3     // 3rd insert- header initial value (8)
4     Node_t* curr = list->header;
5     for (int i = list->level - 1; i >= 0; i--) { // i: 3:0
6         while (curr->forward[i] && curr->forward[i]->value < value)
7             { // Traverse & find insert position based on ascend. order
8                 curr = curr->forward[i];
9             }
10        update[i] = curr;
11    }
12    curr = curr->forward[0];
13    if (curr && curr->value == value) return;
14
15    // continues
```

Skip Lists - 3rd insert

```
1      // assume random level (3) from now less than 4
2      int lvl = random_level();
3      // for 3rd insert, false (3 > 4), skip the if block
4      if (lvl > list->level) {
5          // do nothing
6      }
7      Node_t *n = (Node_t*)malloc(sizeof(Node_t));
8      n->value = value; // update the new value
9      // For first three levels, update the nodes
10     for (int i = 0; i < lvl; i++) {
11         // newNodes point to previous 1st Node
12         n->forward[i] = update[i]->forward[i];
13         // Mark newNode as current first node
14         update[i]->forward[i] = n;
15     }
16 }
```

Skip Lists Hierarchy till 3rd insert

```
1 Level 3: 10
2 Level 2:  7 10
3 Level 1:  7  8 10
4 Level 0:  7  8 10
5
6
7 Level 3:  -  - 10   (Skip List)
8 Level 2:  7  - 10
9 Level 1:  7  8 10
10 Level 0:  7  8 10
```

Skip List - Search

```
1  /* Search key - Continue search one-by-one from top level to bottom */
2  int search(SkipList *list, int key) {
3      Node *current = list->header;
4      for (int i = list->level; i >= 0; i--) {
5          // Search starts with list at top level
6          // More no. of skips, less no. of search option
7          // Search moves to next level list from breakpoint
8          // Good amount. of skips, no. of search option rel. more
9          // Traverse, stop and move to next level from breakpoint
10         // if search key/value exceeds or equals node key
11         while (current->forward[i] && current->forward[i]->key < key)
12             current = current->forward[i]; // new current (while break
13     }
14 }
15 current = current->forward[0];
16 if (current && current->key == key) return 1;
17 return 0;
18 }
```

Skip Lists - Search

► Search for 8

```
1 Level 3: 10                (10 > 8, break)
2 Level 2:  7 10             ( 7 < 8, continue, 10 > 8, break)
3 Level 1:  7  8 10          ( 8 = 8, break)
4 Level 0:  7  8 10          ( 8 = 8, break)
5
6 Final check:
7     breakpoint node value, 8 == target (8)
8     // success (found match)
```

► Note: Search skips also the reason for calling it as Skip List

Skip Lists - delete node

```
1 // This part is traversal (similar to insert)
2 void delete(SkipList *list, int value) { // value = 8
3     Node_t* update[MAX_LEVEL];
4     Node_t* curr = list->header;
5     for (int i = list->level - 1; i >= 0; i--) {
6         while (curr->forward[i] && curr->forward[i]->value < value)
7         {
8             curr = curr->forward[i]; // Traverse
9         }
10        update[i] = curr;
11    }
12    curr = curr->forward[0];
13    // continues
```

Skip Lists - delete node

```
1      // Delete: Node to be deleted, not null and its value matches
2      if (curr && curr->value == value) {
3          for (int i = 0; i <= list->level; i++) {
4              // Identified value-matching-node at each level
5              // If delete node missing, move to next level
6              if (update[i]->forward[i] != curr) break;
7              // curr will be deleted, pass its forward
8              // pointer as previous nodes forward
9              update[i]->forward[i] = curr->forward[i];
10         }
11         free(curr); // delete node 'curr'
12         while (list->level > 0 &&
13             list->header->forward[list->level] == NULL) {
14             // After the delete, if any level is empty
15             // Especially any top-level, remove it
16             list->level--;
17         }
18     }
19 }
```